
ixi-sLM: On-Device sLM for AI Call Agent Service

Han-Sang Lee
LG Uplus

Kyuho Lee
LG Uplus

Juneyoung Park
OptAI Inc.

Seongbae Lee
OptAI Inc.

Jin Kyu Park
LG Uplus

Minjae Kim
LG Uplus

Yuri Hong
OptAI Inc.

Jinwoo Lee
OptAI Inc.

Youngwook Kwon
OptAI Inc.

Seongwan Kim
OptAI Inc.

Seunghoon Hyeon
LG Uplus

Jaeho Lee*
OptAI Inc.

Kiduk Kwon*
LG Uplus

Abstract

Running an AI call agent on-device makes it private, network-independent, and free of per-query cloud cost, but shipping one as a real-world service imposes constraints a cloud deployment never faces: ~ 1 GiB memory budget, in-call latency and thermal limits, static-integer NPU execution, Korean STT input far from sLM pretraining, and cloud-level quality expectations. We bring ixi-sLM, a 1.2B Korean sLM, to mobile NPUs through *on-device modeling*: domain-adaptive modeling trims the vocabulary by call frequency to shrink the embedding and LM head that take nearly half the model under INT4 transformer weights and refits the tokenizer to call speech (18% fewer tokens on call traffic), while task-aware quantization calibrates on real calls and assigns per-module mixed precision by activation sensitivity. A single domain-adapted model runs ixi-O’s two production services, AI Call Summary and AI Call Answering, at 94% of a state-of-the-art cloud LLM’s task-weighted quality; our on-device modeling carries that quality onto the NPU with only a marginal drop in overall score, in an 803 MiB model (28% of FP16) at 23% of the CPU’s per-call energy. The same recipe deploys across Qualcomm and Apple NPUs and serves several million ixi-O subscribers in production.

1 Introduction

Privacy, network independence, and freedom from per-query cloud cost are pushing foundation-model intelligence onto the device [14, 21]; an AI call agent makes each pressure acute, as call content is sensitive, connectivity drops mid-call, and cloud cost scales with call volume.

Shipping such an agent as a real-world service, however, is governed by constraints a cloud deployment never faces. We identify five (3): **C1 (Memory)**, a ~ 1 GiB working-memory budget; **C2 (Latency and thermals)**, in-call responsiveness within a shared thermal envelope; **C3 (Execution model)**, precompiled static-integer NPU graphs committed at export time; **C4 (Input distribution)**, Korean STT traffic, rich in disfluencies and agglutinative morphology, that diverges from sLM pretraining and inflates token counts; and **C5 (Quality)**, direct user comparison against cloud services. No existing path satisfies all five at once: on-device frameworks fix tokenization and quantization as configuration rather than adapting them (C3, C4), and mobile quantization targets general benchmarks rather than service traffic (C4, C5).

We present ixi-sLM, the on-device model that powers ixi-O, an AI call agent used by several million subscribers. Every design decision in it follows from these constraints. Our contributions are: (i)

*Corresponding authors: jaeho.lee@opt-ai.kr, kwonkiduk@lguplus.co.kr

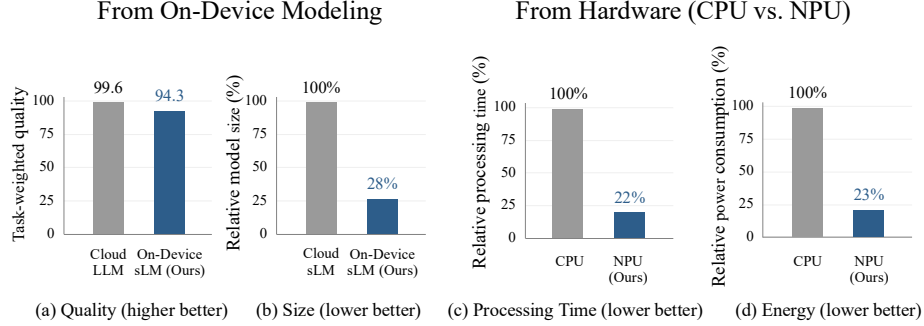


Figure 1: xxi-sLM (blue) compared with a cloud LLM, its FP16 counterpart, and the CPU: (a) 94% of cloud quality, (b) 803 MiB, 28% of FP16, (c) 22% of CPU latency, (d) 23% of CPU energy.

domain-adaptive modeling that trims the vocabulary by call frequency and refits the tokenizer to Korean call speech, cutting 151 MiB from the deployed model and reducing call-traffic token counts by 18% (C1, C4; 4.1); (ii) **task-aware mixed-precision quantization** that calibrates on real calls and assigns per-module bit-width by activation sensitivity, with only a marginal drop in overall score (C3, C4; 4.2); and (iii) **a production deployment in xxi-O** that runs both services on one domain-adapted model, reaching 94% of a SOTA cloud LLM’s quality and largely retaining it through quantization, across Qualcomm and Apple NPUs (Figure 1), evaluated against all five constraints (5).

2 Background and Related Work

Generic on-device LLM Frameworks such as llama.cpp [5], ExecuTorch [16], and ANEMLL [2] execute LLMs on consumer hardware, from CPU/GPU to NPU-specific backends; all expose quantization and tokenization as fixed configuration, not as levers to adapt to a service’s constraints (Appendix A, Table 5).

Domain adaptation for sLMs spans tokenizer-level vocabulary adaptation [9] and knowledge-level domain-adaptive pretraining [7]; a domain-adapted model can outperform a larger general one on Telco tasks [8]. We use *domain-adaptive modeling* for this service-facing adaptation to Korean call traffic.

Quantization for on-device LLMs is dominated by post-training quantization (PTQ), spanning weight-only quantization, activation/weight rebalancing, rotation- or trellis-based outlier suppression, and mobile-oriented quantization [4, 12, 22, 17, 15, 19, 18]; most assume GPU floating-point or per-token statistics that a static-integer NPU does not support well. We use *hardware-aware optimization* for the quantization and export choices these constraints demand; together with domain-adaptive modeling it defines our *on-device modeling*: a service-specific sLM both accurate for its call domain and executable within an NPU budget.

3 xxi-O Service and Deployment Constraints

Services and sLM tasks. Of the functions in the full xxi-O agent (Figure 2, Appendix H), two run on the sLM, each as a task configuration of one shared base model. **AI Call Summary** is triggered post-call: one inference pass turns the speech-to-text (STT) transcript into a *one-line summary*, a *detailed summary*, *keyword extraction*, and *task suggestion* for commitments the user made. **AI Call Answering** acts as an autonomous assistant when the user is unavailable, identifying the caller and their purpose and streaming that context to the user in real time.

Real-world deployment constraints. Turning these services into a shipped product imposes the five constraints of 1 concretely. **C1:** devices span 6–12 GiB of memory shared with the OS and other



Figure 2: Overview of ixi-O, our on-device AI call agent, and its core functions: AI call summary, real-time call script, AI call answering, fraud prevention, and smart call search.

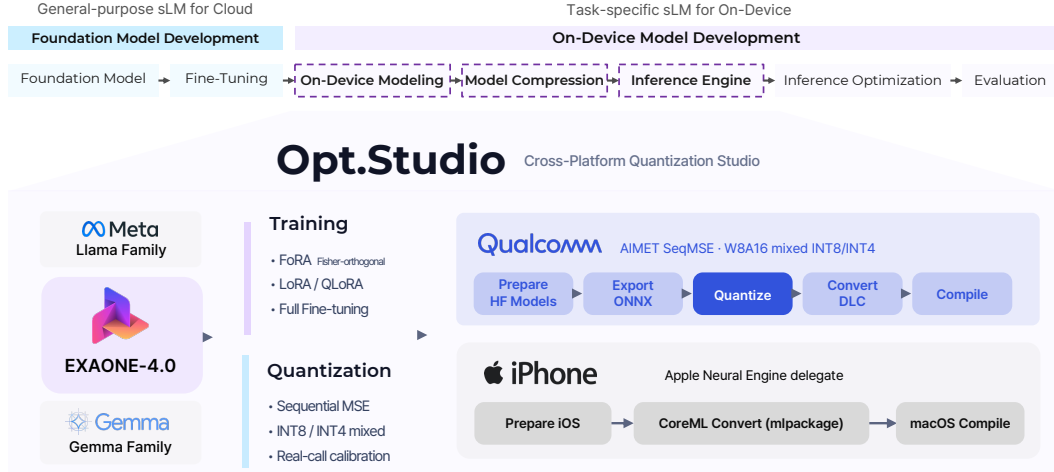


Figure 3: Overview of the on-device sLM converting pipeline. Opt.Studio applies task-aware quantization and then exports the model to Qualcomm and Apple NPUs from a single recipe.

apps, budgeting ~ 1 GiB for the model (Appendix C). **C2:** summarization must complete within seconds of hang-up, answering must stream during a live call, and sustained CPU load throttles thermally. **C3:** mobile NPUs execute precompiled static-integer graphs (fixed shapes, calibration-precomputed activation ranges, a narrow operator and bit-width menu); a deployment commits at export time to a configuration the hardware executes natively, and any mismatch surfaces only on device. **C4:** inputs are Korean STT transcripts with recognition errors, fillers, and missing punctuation, far from sLM pretraining text. **C5:** output quality must stand comparison with the cloud services users already know.

Consequence: model selection. C1 and C2 rule out models above 3B on mobile [14, 1], pointing to a 1B-class model; C4 requires strength in Korean, where English-centric sLMs lose ground (Appendix C). We therefore build ixi-sLM on EXAONE4.0 1.2B [10], pretrained on large-scale Korean.

Consequence: no generic path suffices. Before building our own pipeline, we tried to deploy the same Korean-native 1.2B sLM through existing NPU paths. LiteRT [6] failed to compile the model for NPU execution; ExecuTorch’s QNN delegate exported and compiled it, and the graph ran correctly in PyTorch, yet on device its static-integer outputs were unusable, even though both frameworks run Llama-family models on the same hardware [16]. The ExecuTorch figures in Table 7 therefore reflect speed only, not usable output. C3 is thus not satisfiable by configuration alone, directly motivating 4.2 and the framework below.

Cross-platform framework. The model must be adapted to Korean call traffic (C4), quantized for static-integer NPUs (C3), and exported consistently across Qualcomm and Apple backends. We therefore built an in-house cross-platform studio carrying one trained sLM to both NPU targets from a single recipe (Figure 3): it accepts EXAONE, Llama, or Gemma-family base models, applies domain training and task-aware quantization once, and exports each revision through the corresponding backend compiler, keeping both deployments numerically consistent.

4 On-Device Modeling

This section describes how we build ixi-sLM to satisfy 3’s constraints: domain-adaptive modeling (4.1) targets the input distribution (C4) and memory budget (C1); task-aware quantization (4.2) targets the NPU execution model (C3) while preserving quality (C5).

4.1 Domain-Adaptive Modeling

Domain-adaptive modeling comprises (i) *domain training*, that is, continued training and task tuning on de-identified call corpora [7], producing the FP16 *Cloud sLM* of 5 (Appendix B); and (ii) the tokenizer-level adaptation below, the footprint-relevant component we detail.

Vocabulary trimming. EXAONE4.0 1.2B carries a 102,400-token vocabulary; at hidden size 2,048 its tied embedding and LM head occupy 419 MiB in FP16. Under the on-device configuration this class of model assumes, an FP16 embedding beside INT4 transformer weights, this single tensor accounts for 44% of the model. The imbalance is structural: quantization shrinks the transformer weights but leaves the embedding in place, so its share grows as the rest contracts (Figure 4a).

Table 1: Merging accounts for the entire tokenization gain; trimming acts purely on footprint. Sizes in MiB.

Config.	Vocab.	Tok/call	Δ	Size (MiB)
Original	102,400	1,324	–	954
Merge only	102,400	1,087	–17.9%	954
Trim only	65,536	1,326	+0.2%	803
Both	65,536	1,090	–17.7%	803

Vocabulary trimming attacks this footprint directly (C1) by removing tokens that never appear at inference: language-based trimming drops tokens outside the target languages, effectively lossless [20], and a frequency-based step removes rare tokens within them. The frequency cutoff is conventionally drawn over a general corpus, unsafe for a vocabulary already tuned to Korean: tokens rare in general text may be load-bearing in the domain. We instead redraw the cutoff over *real call data*, where a token absent from call traffic is genuinely dead for the service, licensing a more aggressive cut. The two stages reduce the vocabulary from 102,400 to 65,536 tokens (36%), lowering the tied embedding and LM head from 419 MiB to 268 MiB, the embedding share from 44% to 33% (Figure 4b), and the total model from 954 MiB to 803 MiB (15.8%).

Domain-aware tokenizer. EXAONE4.0 also ships with reserved slots its vocabulary never assigns; we reuse them to adapt the tokenizer by *merging*, rather than removing, tokens, holding the vocabulary at 65,536 so the adaptation adds no footprint (C1, C4). On the input side, frequent Korean call expressions (greetings, acknowledgments, identified by frequency over call transcripts) collapse from several BPE tokens into one, shortening STT input; on the output side, fixed strings the service emits on every call (e.g., the section markers "One Line Summary:", "Detailed Summary:") are merged so a header costs a single decoding step.

The merges compound over a full call: across held-out calls, adaptation cuts token counts by 18% (Table 1), and the reduction is what lets even the longest calls fit an entire transcript and its structured output in a single fixed window.

4.2 Task-Aware Quantization

Effective quantization for mobile NPUs must match both the input distribution (C4) and the hardware execution model (C3): PTQ estimates activation ranges from calibration data, so clean text such as WikiText can misrepresent STT-based call transcripts (recognition errors, fillers, missing punctuation), while global INT4 can over-compress sensitive modules. Our strategy combines real-call calibration,

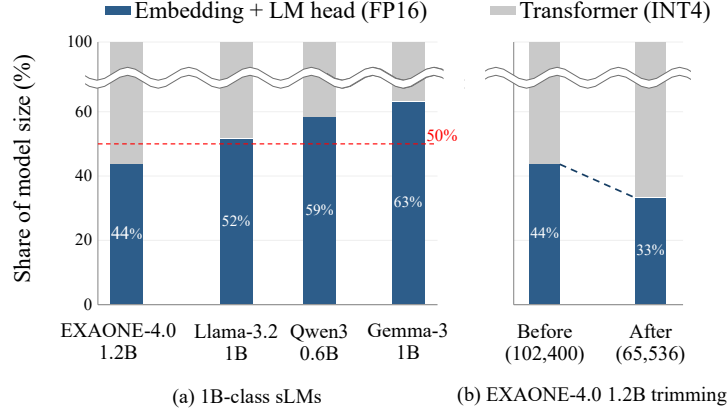


Figure 4: (a) Embedding (FP16) vs. transformer (INT4) share across 1B-class sLMs. (b) shows the same FP16-embedding accounting as (a), isolating the effect of trimming; the deployed model quantizes the embedding further (4.2).

SeqMSE-based module-wise mixed precision, and top-k probability alignment for operating-point selection (details in Appendix D).

Real-call calibration. We compare WikiText and real-call calibration under W4A16 (4-bit weights, 16-bit activations), W8A16, and MW4A16 (W8A16 with low-sensitivity modules reduced to 4 bits via SeqMSE), evaluating with ROUGE-L [11] between GPU (original) and NPU (quantized) outputs. Real-call calibration achieves higher ROUGE-L under all three settings (Table 2), with the largest gain (+2.76) in MW4A16: task-aligned calibration preserves call-agent behavior more effectively than clean text. ROUGE-L here isolates the *calibration-data* effect within each setting; the choice *among* settings is made by distribution alignment and final task accuracy below.

Table 2: Real-call vs. WikiText calibration per quantization setting. Sizes are after vocabulary trimming; MW4A16 is the mixed-precision setting initialized from W8A16.

Quant.	Size (MiB)	Bits	ROUGE-L (%)	
			Call	WikiText
W4A16	726	4.10	80.63	79.42
MW4A16	803	5.09	82.23	79.47
W8A16	1,111	8.10	84.29	81.67

Mixed-precision quantization. Modules in the same transformer layer respond very differently to low precision. Measuring each linear projection’s sensitivity with Sequential MSE (SeqMSE), the activation difference between the original and quantized module on calibration data, we find `o_proj` and `down_proj` consistently far less sensitive than the other projections (Appendix D, Figure 5); mixed precision therefore keeps high-SeqMSE modules at INT8 and lowers only low-SeqMSE modules to INT4.

Top-k probability alignment. To pick the operating point we compare per-layer top-k token distributions of the original and quantized models via the logit lens [3]. The 803 MiB MW4A16 model preserves the final-layer alignment of the 1,111 MiB W8A16 model, whereas global W4A16 loses alignment despite its competitive ROUGE-L in Table 2 (Figure 6); with the task accuracy of 5.2, this fixes MW4A16 as the deployment point.

5 Experiments

5.1 Experimental Setup

We evaluate against the constraints of 3: quality (C5) via task accuracy, C1–C3 via on-device measurements, comparing *Cloud LLM* (a frontier commercial LLM, the quality reference), *Cloud sLM* (our domain-adapted EXAONE4.0 1.2B [10] in FP16, 4.1), and *ixi-sLM* (the same model

quantized, 4.2), on 100 de-identified held-out calls (up to 4K tokens) from the two ixi-O services, on a Galaxy S25 Ultra (Snapdragon 8 Elite) and an iPhone 17 Pro (A19 Pro), on CPU and NPU. Task accuracy is a weighted rubric score (0–100) graded by three frontier LLM judges under cross-evaluation [23, 13] (rubric in Appendix E); on-device performance covers throughput, size, latency, and energy (Appendices F, G), with latency, energy, and thermal results measured on the Samsung Galaxy S25 Ultra.

5.2 Task Accuracy

Table 3 separates two gaps toward C5. From the cloud LLM to our FP16 sLM, domain training (continued training and task tuning, Appendix B) reaches 94.38 and 95.70, or 94.8% and 96.7% of the cloud LLM at a fraction of its size; from the FP16 model to ixi-sLM, quantization costs only 0.05 and 0.30 points, concentrated in task suggestion, a conditional sub-task evaluated on few calls (Appendix E). The drop is larger for schedule and location suggestion (99.55→87.05 and 97.61→83.95 in relative score), but the app only proposes these actions and the user confirms them (Figure 9a).

Table 3: Task accuracy via LLM judge (weighted rubric, 0–100). Cloud sLM: our domain-adapted FP16 model; ixi-sLM: the same model quantized (<1 GiB).

	Summary	Answering
Cloud LLM	99.60	99.00
Cloud sLM	94.38	95.70
ixi-sLM (Ours)	94.33	95.40

5.3 On-Device Performance

Our on-device modeling shrinks the model from its 2.9 GiB FP16 checkpoint to 803 MiB (28%) and cuts token counts by 18%, meeting the C1 budget while fitting a full call in a single fixed window; the NPU cuts latency to 22% (4.5×) and per-call energy to 23% (Table 4), and over a 300 s sustained run stays below 35 °C in surface temperature, beneath the 36 °C throttling threshold, while the CPU crosses that threshold within 60 s and plateaus near 45 °C, meeting C2. The saving concentrates in compute-bound prefill (~17× less energy) versus memory-bound decode (~2×; Appendix G). Against other Snapdragon-NPU runtimes (Appendix F), our engine is ~1.5×/1.7× faster than ExecuTorch on prefill/decode, over 30× faster than llama.cpp on prefill, in a 22% smaller model than ExecuTorch.

Table 4: Per-call latency and energy, CPU vs. NPU.

	Latency (s)	Energy (mAh)
CPU (ExecuTorch)	45	7.79
NPU (ixi-sLM)	10	1.79

6 Conclusion

We presented ixi-sLM, the on-device model behind the ixi-O call agent, built by treating real-world deployment constraints (C1–C5) as the starting point of the design: domain-adaptive modeling and task-aware quantization follow from them, and one 1.2B model, at 94% of a SOTA cloud LLM’s quality, runs across Qualcomm and Apple NPUs with only a marginal drop in overall score. Future work: inference optimization, on-device personalization, broader evaluation (Appendix J).

References

- [1] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C. Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a flash: Efficient large language model inference with limited memory. *arXiv preprint arXiv:2312.11514*, 2023.
- [2] ANEMLL. ANEMLL: Artificial neural engine machine learning library. <https://github.com/Anemll/Anemll>, 2025. Accessed: 2026-06-13.
- [3] Nora Belrose, Zach Furman, Logan Smith, Danny Halawi, Igor Ostrovsky, Lev McKinney, Stella Biderman, and Jacob Steinhardt. Eliciting latent predictions from transformers with the tuned lens. *arXiv preprint arXiv:2303.08112*, 2023.

- [4] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *ICLR*, 2023.
- [5] Georgi Gerganov et al. llama.cpp. <https://github.com/ggml-org/llama.cpp>, 2023.
- [6] Google AI Edge. LiteRT: High-performance runtime for on-device AI. <https://ai.google.dev/edge/litert>, 2024.
- [7] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don’t stop pretraining: Adapt language models to domains and tasks. In *ACL*, 2020.
- [8] Seonwu Kim, Yohan Na, Kihun Kim, Hanhee Cho, Geun Lim, Mintae Kim, Seongik Park, Ki Hyun Kim, Youngsub Han, and Byoung-Ki Jeon. Efficient industrial sLLMs through domain adaptive continual pretraining. *arXiv preprint arXiv:2507.06795*, 2025.
- [9] Seungduk Kim, Seungtaek Choi, and Myeongho Jeong. Efficient and effective vocabulary expansion towards multilingual large language models. *arXiv preprint arXiv:2402.14714*, 2024.
- [10] LG AI Research. EXAONE 4.0: Unified large language models integrating non-reasoning and reasoning modes. *arXiv preprint arXiv:2507.11407*, 2025.
- [11] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, 2004.
- [12] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. In *Proceedings of Machine Learning and Systems*, volume 6, 2024.
- [13] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023.
- [14] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunsang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. MobileLLM: Optimizing sub-billion parameter language models for on-device use cases. *arXiv preprint arXiv:2402.14905*, 2024.
- [15] Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. SpinQuant: LLM quantization with learned rotations. In *ICLR*, 2025.
- [16] PyTorch Team. ExecuTorch: A unified runtime for on-device AI. <https://github.com/pytorch/executorch>, 2024. Accessed: 2026-05-28.
- [17] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. OmniQuant: Omnidirectionally calibrated quantization for large language models. In *ICLR*, 2024.
- [18] Fuwen Tan, Royson Lee, Łukasz Dudziak, Shell Xu Hu, Sourav Bhattacharya, Timothy Hospedales, Georgios Tzimiropoulos, and Brais Martinez. MobileQuant: Mobile-friendly quantization for on-device language models. In *Findings of EMNLP*, 2024.
- [19] Albert Tseng, Qingyao Sun, David Hou, and Christopher De Sa. QTIP: Quantization with trellises and incoherence processing. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [20] Asahi Ushio, Yi Zhou, and Jose Camacho-Collados. Efficient multilingual language model compression through vocabulary trimming. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 14725–14739, Singapore, 2023. Association for Computational Linguistics.

- [21] Haoxin Wang, Xiaolong Tu, Hongyu Ke, Huirong Chai, Dawei Chen, and Kyungtae Han. lm-Meter: Unveiling runtime inference latency for on-device language models. *arXiv preprint arXiv:2510.06126*, 2025.
- [22] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *ICML*, 2023.
- [23] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.

A Generic On-Device Framework Comparison

llama.cpp [5] runs GGUF-quantized models on CPU and GPU and exposes an experimental Snapdragon NPU that executes but delivers an order of magnitude lower prefill throughput than delegating runtimes; it provides no NPU path for our Apple target. ExecuTorch [16] compiles PyTorch models to mobile backends with partial NPU delegation via QNN, and ANEMLL [2] compiles exclusively for the Apple Neural Engine. All three treat quantization and tokenization as fixed configuration chosen from a menu, rather than as levers to adapt to a service’s input distribution and hardware target; this gap motivates our platform (3).

Table 5: Comparison of generic on-device LLM frameworks. “Modeling” denotes whether the framework exposes tokenizer/quantization adaptation as first-class levers (○: supported, △: partial, ×: unsupported).

Framework	OS	Modeling	Hardware		
			CPU	GPU	NPU
ExecuTorch	AOS/iOS	△	○	○	△
llama.cpp	AOS/iOS	×	○	○	△
ANEMLL	iOS	×	○	×	○

B Domain Training

The *Cloud sLM* of 5 is produced by domain training the EXAONE4.0 1.2B base model on de-identified call-domain corpora before any tokenizer or quantization step: continued training on call-domain text aligns the model with STT-style input (disfluencies, fillers, missing punctuation), and task-specific tuning on the two service formats (AI Call Summary and AI Call Answering) teaches the structured output schemas shown in Appendix I. Training follows standard domain-adaptive practice [7]; all corpora are de-identified before use, and evaluation calls (5.1) are held out from training.

C Device Chip Tiers and Base-Model Selection

C.1 Mobile hardware envelope

Target devices span 6–12 GiB of system memory and NPUs delivering roughly 20–50 TOPS across product tiers. A deterministic on-device deployment must coexist with the OS and other applications, so we budget approximately 1 GiB of working memory for the model. Combined with the latency and thermal headroom an active call requires, which rules out models above 3B, this envelope points to a 1B-class model.

C.2 Korean capability of English-centric sLMs

Within the 1B-class budget, the base model must also be strong in Korean. English-centric sLMs, including LLaMA3.2 1B, Gemma3 1B, and Qwen2.5 1.5B, score competitively on general English benchmarks but lose ground in Korean: their generic BPE vocabularies fragment Korean’s agglutinative morphology into long token sequences, inflating the context window and degrading quality under a fixed budget. EXAONE4.0 1.2B, pretrained on large-scale Korean, tokenizes the same text more compactly and serves as our base model.

D Quantization Analysis

This section provides the module-level evidence used to choose the MW4A16 mixed-precision setting in 4.2. We use two diagnostics. First, SeqMSE measures how much each projection module’s activation changes when it is quantized, using the same real-call calibration distribution used for PTQ. Second, top-k probability alignment measures whether a quantized model preserves the token distribution of the original model across late layers and at the final prediction.

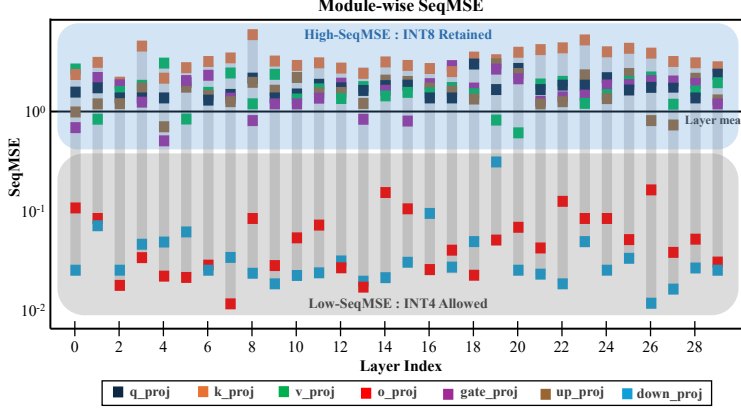


Figure 5: Module-wise SeqMSE on EXAONE4.0 1.2B (W8A16). Modules above the threshold (blue region) are kept at INT8, while those below it (gray region) are lowered to INT4.

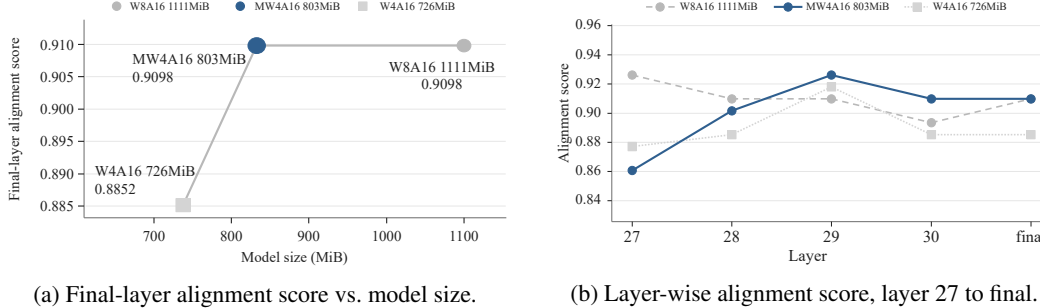


Figure 6: Alignment score comparison across quantized model sizes. (a) shows the final-layer alignment score as a function of model size, where the 803 MiB model preserves the final-layer alignment of the 1,111 MiB W8A16 model at a smaller size. (b) shows the layer-wise alignment score in the late layers, indicating how each quantized model follows the original model before the final prediction.

Figures 5 and 6 motivate MW4A16 as the deployment setting: because the o_proj and down_proj modules consistently exhibit lower SeqMSE, quantizing only these low-sensitivity modules to INT4 reduces the model footprint to 803 MiB while retaining the final-layer alignment of W8A16; in contrast, globally applying INT4 in W4A16 degrades alignment. Thus, MW4A16 satisfies the on-device memory budget while preserving the original model’s prediction behavior more closely than global 4-bit quantization.

E Task Accuracy: Full Rubric

Table 6: Detailed task-accuracy rubric for both services. For each sub-task we report the raw judge score (Raw), its relative score against a per-sub-task reference, and the weighted contribution to the total (Fin. = $Wt \times Rel.$). The reference is the best score observed across systems and judges, so it can exceed the highest value shown. Raw scales differ across sub-tasks. Scores are averaged over a three-LLM judge panel under cross-evaluation.

Sub-task	Wt.	Cloud LLM			Cloud sLM			ixi-sLM		
		Raw	Rel.	Fin.	Raw	Rel.	Fin.	Raw	Rel.	Fin.
Service 1: AI Call Summarization										
One-line summary	25%	4.77	100.00	25.00	4.61	96.65	24.16	4.66	97.69	24.42
Detailed summary	50%	22.00	100.00	50.00	21.42	97.36	48.68	21.48	97.64	48.82
Keyword extraction	10%	4.81	100.00	10.00	4.67	97.09	9.71	4.73	98.34	9.83
Task suggestion (schedule)	6%	4.22	95.91	5.75	4.38	99.55	5.97	3.83	87.05	5.22
Task suggestion (location)	6%	4.49	97.40	5.84	4.50	97.61	5.86	3.87	83.95	5.04
Task suggestion (contact)	3%	3.00	100.00	3.00	0.00	0.00	0.00	1.00	33.33	1.00
Total	100%			99.60			94.38			94.33
Service 2: AI Call Answering										
Intent classification	30%	5.00	100.00	30.00	4.86	97.20	29.16	4.85	97.00	29.10
Information extraction	30%	4.90	98.00	29.40	4.81	96.20	28.86	4.79	95.80	28.74
Guardrail	20%	5.00	100.00	20.00	4.98	99.60	19.92	4.98	99.60	19.92
Naturalness	20%	4.90	98.00	19.60	4.43	88.60	17.72	4.41	88.20	17.64
Total	100%			99.00			95.70			95.40

Table 6 reports the full per-sub-task rubric behind Table 3. Each service’s score is a weighted aggregate of sub-task scores; for each sub-task we report the raw judge score, its relative score against the per-sub-task reference, and its weighted contribution. Scores are averaged over a panel of three frontier LLM judges under cross-evaluation to reduce single-judge and self-preference bias. Conditional sub-tasks (task suggestion) fire on only a subset of the 100 evaluation calls, which is why their scores are noisier than sub-tasks evaluated on every call.

F On-Device Performance: Speed and Model Size

Table 7: Prefill and decode throughput (token/sec) and model size (MiB) across OS, Hardware, and Frameworks. Dashes mark unsupported configurations. All models were benchmarked with Max 4K Tokens (3K prompt tokens and 0.5~1K generated tokens)

OS	Hardware	Framework	LLaMA3.2 1B			EXAONE4.0 1.2B		
			Prefill	Decode	Size	Prefill	Decode	Size
Test Device 1: Samsung Galaxy S25 Ultra (Snapdragon 8 Elite)								
Android	CPU	llama.cpp	195.0	40.0	728 MiB	49.4	39.4	786 MiB
		ExecuTorch	526.4	66.3	821 MiB	145.7	54.2	853 MiB
	NPU	llama.cpp	132.4	25.5	851 MiB	97.2	17.1	887 MiB
		ExecuTorch	2639.5	42.6	1,037 MiB	2327.1	32.5	1,026 MiB
		Ours	5025.1	64.7	980 MiB	3558.2	55.8	803 MiB
Test Device 2: Apple iPhone 17 Pro (A19 Pro)								
iOS	CPU	llama.cpp	356.2	24.7	763 MiB	-	-	-
		ExecuTorch	160.3	25.7	822 MiB	-	-	-
	NPU	llama.cpp	-	-	-	-	-	-
		ExecuTorch	-	-	-	-	-	-
		ANEMLL	418.2	27.9	729 MiB	-	-	-
		Ours	485.1	32.1	735 MiB	412.2	19.5	783 MiB

Table 7 reports prefill/decode throughput (tokens/s) and model size for EXAONE4.0 1.2B and LLaMA3.2 1B on the Samsung Galaxy S25 Ultra (Qualcomm Snapdragon 8 Elite) and Apple iPhone 17 Pro (A19 Pro), across CPU and NPU and across ExecuTorch, llama.cpp, and our platform. For frameworks other than ours, models were converted using the stable conversion options provided by each open-source framework and run with the default inference options of the corresponding runtime. On the Snapdragon NPU ixi-sLM occupies 803 MiB against ExecuTorch’s 1,026 MiB, a 22% reduction combining vocabulary trimming with module-wise mixed precision.

G On-Device Performance: Power and Thermal

G.1 Power

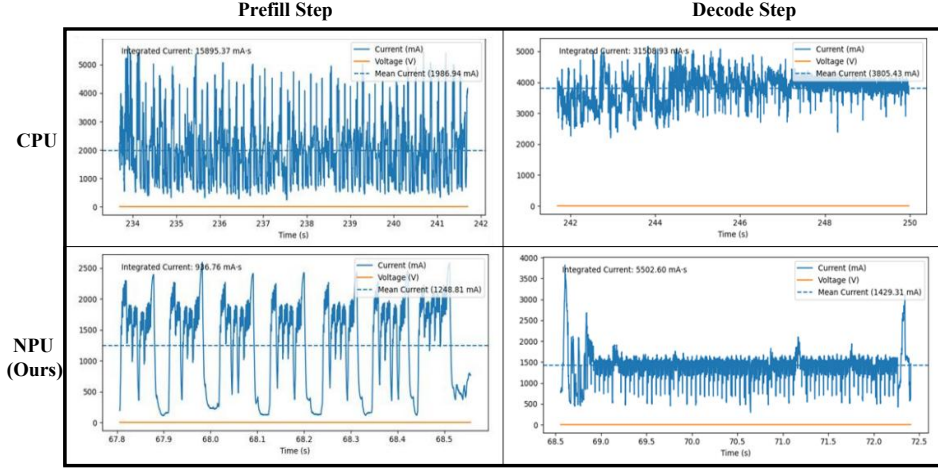


Figure 7: Battery current traces over one representative call, CPU vs. NPU, segmented into prefill and decode phases.

We measure on-device power with a power monitor (Monsoon Power Monitor) sampling battery current at 5 kHz against a 5,000 mAh, ~ 4.2 V reference. The target model (ixi-sLM) serves one representative call (~ 4 K tokens), segmented into prefill and decode phases. Table 8 breaks the per-call energy of Table 4 down by stage: the NPU’s saving is concentrated in the compute-bound prefill phase ($\sim 17\times$), with a $\sim 2\times$ saving in the memory-bound decode phase.

Table 8: Per-call Energy (mAh) by phase, CPU vs. NPU.

Stage	Prefill	Decode	Total
CPU	4.42	3.37	7.79
NPU	0.26	1.53	1.79

G.2 Thermal

Sustained inference must also respect the device’s thermal envelope: a call agent that throttles mid-call fails its latency contract. Table 9 reports device temperature at 30 s intervals over a 300 s sustained run.

The CPU crosses the 36 °C throttling threshold within the first 60 s, climbs to ~45 °C, and plateaus there for the remainder of the run, while the NPU peaks at 34.5 °C and stays under the threshold throughout, leaving thermal headroom for the rest of the system during an active call.

Table 9: Temperature (°C) at 30 s intervals over a 300 s sustained run, CPU against NPU. The final column is the average.

	0	30	60	90	120	150	180	210	240	270	300	Average
CPU	29.9	36.5	43.0	43.5	44.0	44.5	45.0	44.5	44.0	44.6	45.3	42.3
NPU (ixi-sLM)	29.9	30.4	31.6	32.2	33.0	33.1	33.3	33.5	33.5	34.0	34.5	32.6

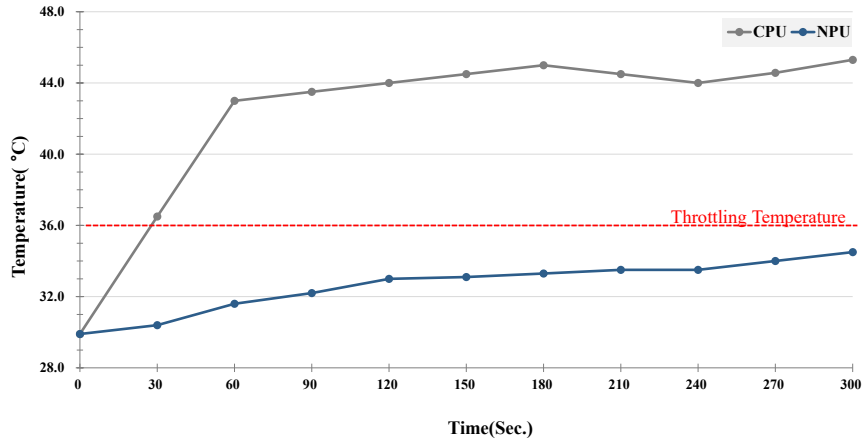


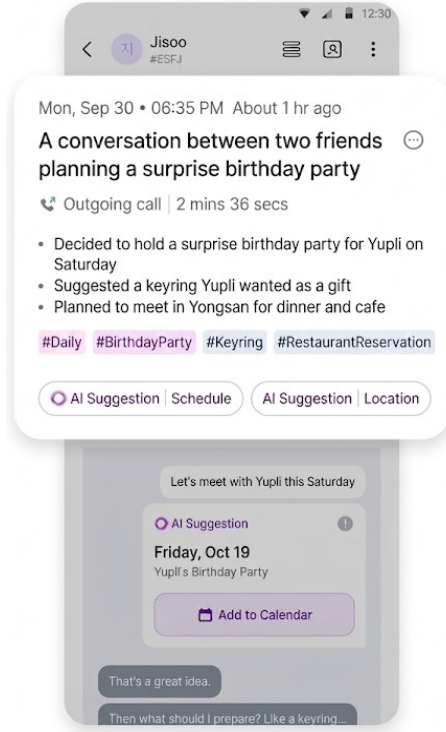
Figure 8: Device surface temperature over a 300 s sustained inference run, CPU vs. NPU. The dashed line marks the 36 °C throttling threshold.

H ixi-O Service Overview

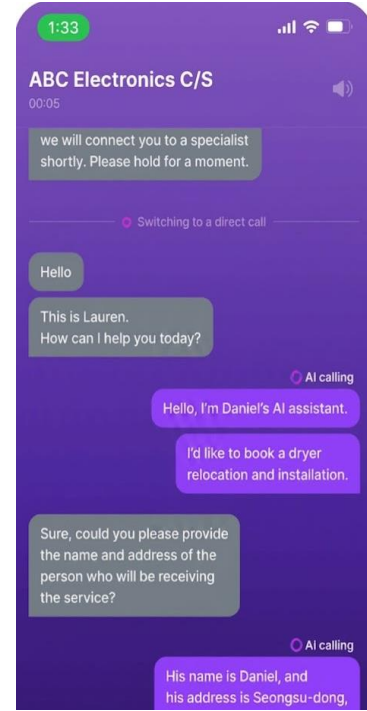
AI Call Summary runs after a call ends: it condenses the conversation into a one-line and a detailed summary with keywords, and proposes follow-up tasks such as calendar events, reminders, and contacts.

AI Call Answering runs when the user is away: the model answers on the user’s behalf and converses with the caller in real time while the user follows the exchange.

Figure 9 shows both experiences; the sub-tasks evaluated in the task-accuracy rubric (Appendix E) correspond directly to these service functions.



(a) Call Summary



(b) AI Call Answering

Figure 9: **(a) AI Call Summary:** After a call ends, the agent produces an on-device summary capturing the conversation’s intent and key points, classifies it with semantic tags, and surfaces actionable suggestions (e.g., adding an event to the calendar). **(b) AI Call Answering:** The agent autonomously carries out a voice conversation on behalf of the user, handling task-oriented dialogue with an external party to fulfill the user’s request.

I Service Prompt

This appendix shows the input and output prompt format used by ixi-sLM. The example below is a synthetic illustration created for exposition only; it was not used for model training, calibration, evaluation, or production inference.

(Korean) Call Summarization Input Prompt

Call Time

2025-04-18 10:30:32

Call Transcript

TX: 안녕하세요 상무님 논의 사항 있어서 전화드렸습니다. 나노기술 연구에서 최근 어떤 진전이 있었나요.

RX: 네 부팀장님도 안녕하세요. 최근에 나노소재를 이용한 신약 개발에 큰 진전이 있었습니다.

TX: 그 신약 개발 프로젝트에 대해 좀 더 자세히 설명해 주시겠어요.

RX: 네. 나노입자를 이용해 약물 전달 효율을 크게 향상시켰습니다.

TX: 그럼 이 기술이 상용화되기까지 얼마나 걸릴까요.

RX: 현재 임상 시험 중이며 상용화까지는 약 2년 정도 예상됩니다.

TX: 임상 시험 결과는 어떤가요 긍정적인가요.

RX: 네 지금까지의 결과는 매우 긍정적입니다. 부작용도 거의 없습니다.

...

(Korean) Call Summarization Output

한줄요약

나노 기술 개발 관련 논의 완료

상세요약

나노 기술 연구 진행 상황에 대해 문의함.

나노 소재를 이용한 신약 개발 진전 상황을 설명함.

신약 상용화까지 2년 예상되며 임상 시험 결과는 매우 긍정적이라고 보고함.

...

키워드

#업무, #회의, #나노기술

AI Task Suggestion

task: schedule

task_title: 프로젝트 회의

date: 2025-04-22

time: 14:00:00

reference: 20

...

(English) Call Summarization Input Prompt

Call Time

2025-04-18 10:30:32

Call Transcript

TX: Hello, Director. I called because there are some matters to discuss. What recent progress has been made in nanotechnology research?

RX: Hello, Deputy Team Lead. Recently, there has been significant progress in developing new drugs using nanomaterials.

TX: Could you explain that new drug development project in more detail?

RX: Yes. We have greatly improved drug-delivery efficiency using nanoparticles.

TX: Then how long will it take for this technology to be commercialized?

RX: It is currently in clinical trials, and commercialization is expected to take about two years.

TX: How are the clinical trial results? Are they positive?

RX: Yes, the results so far are very positive. There have been almost no side effects.

...

(English) Call Summarization Output

One-line Summary

Discussion on nanotechnology development completed

Detailed Summary

Asked about the progress of nanotechnology research.

Explained progress in new drug development using nanomaterials.

Reported that commercialization is expected in two years and that clinical trial results are very positive.

...

Keywords

#work, #meeting, #nanotechnology

AI Task Suggestion

task: schedule

task_title: Project meeting

date: 2025-04-22

time: 14:00:00

reference: 20

...

J Limitations

Our task-accuracy scores rely on an LLM judge panel, and because one judge belongs to the same model family as the cloud baseline, a residual self-preference may inflate that baseline’s scores; cross-evaluation and averaging only partially offset it. The evaluation is also narrow in scope: two services in a single Korean call domain, scored on 100 held-out calls, so the absolute numbers should be read as evidence for this production deployment rather than as a general benchmark. Our results further hold at one operating point, a 1.2B model under a ~ 1 GiB budget on two current NPUs; generalization across hardware is an active engineering effort, and the same recipe is now being extended to eight chipsets. Finally, quality is hardest to measure on conditional sub-tasks such as task suggestion, which are scored on few calls and are therefore sensitive to case frequency rather than to a uniform quantization effect; larger, balanced samples are needed to establish their true on-device quality.